
第 11 章

案例实战：从修 bug 到端到端项目

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

11. 案例实战：从修 bug 到端到端项目

本页是第二部分第 11 章的章首页，也是**全部分练习（exercises）与上机（lab）的统一入口**。四个案例由浅入深，每一节都包含：任务卡、给 Agent 的提示词原文、执行轨迹（真实运行记录）、可执行验收标准、人工复核点、离线替代路径。

本章概览

方法只有在真实任务里才站得住。本章用四个递进案例把第 9、10 章的概念与方法跑一遍，全部复用第 1–8 章的知识作为**验收判据**：

案例	主题	复用章节	能力层级
01 案例一	读懂并修复 NumPy 代码	第 1 章	L1 → L2
02 案例二	脏数据清洗与可视化流水线	第 4、5 章	L3
03 案例三	可复现实验与自动验证	第 3、7 章	L4
04 案例四	端到端小项目与报告	第 0、6、8 章	L5
05 变体、对抗与迁移	四个案例的进阶与对抗实验	全部	进阶
06 实验记录与成本	记录表、成本估算与省钱开关	——	工程

学习目标

- 能用最小复现定位“看起来都对”的静默数值错误；
- 能把清洗策略写成可验收的判定（行数守恒、缺失与异常处理留痕）；
- 会固定种子并做重跑一致性检查，报告里区分“显著”与“解释力强”；
- 能用 Pipeline 与结构检查排除数据泄漏，并让评审代理逐条回应质疑；
- 会用 lab/ 的离线材料在没有 API Key 的环境里完成同样的审查训练。

资源导航

资源	位置	说明
练习入口	exercises/README.md	quiz（10 题）+ assignment（12 题）+ 答案
上机入口	lab/README.md	三段式 lab：离线自检 / 联网任务 / 综合项目
演示数据	lab/data/	scores.csv、scores_dirty.csv、sensor.csv、scores_buggy.py
参考产物	lab/sample_agent_output/	真实运行产物与脱敏会话记录（断网时用）
参考资料	references.md	案例涉及的数据与工具文档
教学说明	teaching.md	教师用：演示脚本、评分 rubric、断网兜底

01 案例一：读懂并修复 NumPy 代码

案例定位：L1 → L2（先只读分析，再单文件修复 + 自测）。复用第 1 章（axis、广播、形状判据）；真实运行记录见 lab/sample_agent_output/。

案例背景

课程练习脚本 `scores_buggy.py` 能正常运行、输出的数字看起来也合理，但它有两处**静默错误**：

- `scores.mean(axis=0)` 求的是“每门课的平均分”（3 个数），学员期望的是“每个学生的平均分”（8 个数）；
- `(scores - scores.min()) / (scores.max() - scores.min())` 用的是**全矩阵**的最小/最大值，把三个科目混在一起归一化，导致每列的范围不再是 0 到 1。

另外它打印的“加权总分前 3 名”其实是“前 3 个学生”，不是真正的排名。

数据：lab/data/scores.csv（8 名学生 × 平时 / 期中 / 期末，满分 100）；**待修脚本**：lab/data/scores_buggy.py。

任务卡（六件套）

要素	内容
目标	找出并修复 <code>scores_buggy.py</code> 的错误，产出 <code>scores_fixed.py</code>
约束	只新增 <code>scores_fixed.py</code> ；不修改 <code>data/scores.csv</code> 与 <code>scores_buggy.py</code>
验收标准	① 平均分形状为 (8,) 且与手算一致；② 逐列归一化后每列 min=0、max=1；③ 加权总分第一为 王五 91.4
证据	运行 <code>python scores_fixed.py</code> 的完整输出 + 三条 <code>assert</code> 通过
复核	原始数据是否被改动、归一化是否逐列、排名是否真排序
提交	脚本 + 运行输出 + 会话记录

给 Agent 的提示词（原文）

工作区里有一个成绩分析脚本 `data/scores_buggy.py` 和配套数据 `data/scores.csv`。请只读分析，不要修改任何文件：

- 1) 指出脚本里每个隐蔽错误，说明它算出来的到底是什么、为什么错；
- 2) 给出一个最小复现命令（能证明错误的形状或数值）；
- 3) 给出修正方案（关键行改成什么）；
- 4) 为修正后的结果写 3 条可执行断言（形状、逐列归一化范围、加权总分最高分是谁）。

请用简洁的条目输出。

Agent 执行轨迹（示例，基线 v0.1.5-rc.2 / 2026-09-19）

第一步：读。它先列出工作区、读脚本与 CSV，确认列名与行数（而不是直接给答案）。推理摘录：

The lab directory only has `scores_buggy.py` as a `.py` file ... the buggy script reads `data/scores.csv`.

Let me read scores.csv to understand the structure and check who 王五 is and confirm columns.

第二步：判。它给出的问题清单（要点）：

位置	它算出来的	应该是什么
student_mean = scores.mean(axis=0)	每门课的平均分，3 个数	每个学生的平均分，8 个数 (axis=1)
norm = (scores - scores.min()) / ...	全矩阵归一化：各列 min=[0, 0.25, 0.175]、max=[1, 0.975, 0.925]	逐列归一化：各列 min 全 0、max 全 1
total[:3]	前 3 个学生	按总分降序取前三名

第三步：做 + 看。它生成 scores_fixed.py、运行、把输出贴回来（真实输出）：

每个学生的平均分（共 8 个）：

张三 83.33
李四 65.00
王五 91.67
赵六 70.00
钱七 83.00
孙八 61.67
周九 91.33
吴十 76.67

归一化矩阵形状：(8, 3)
逐列归一化范围：min=[0. 0. 0.] max=[1. 1. 1.]
加权总分前 3 名：
第 1 名 王五 加权总分 91.40
第 2 名 周九 加权总分 90.90
第 3 名 钱七 加权总分 82.90
全班平均：77.83
自检通过：形状 / 逐列归一化范围 / 加权总分最高分为王五

说明：Agent 还主动指出“原脚本的 total[:3] 不是排名”，并把打印口径一起修正——这是“读 → 判 → 做 → 看”完整闭环的体现。示例输出会随版本与模型变化；**可复现的是验收标准与断言，不是这段文字。**

修复要点（关键行）

```
# 文件: data/scores_fixed.py (节选)
student_mean = scores.mean(axis=1) # 每个学生一个值
norm = (scores - scores.min(axis=0)) / (scores.max(axis=0) - scores.min(axis=0)) # 逐列归一化
total = scores @ weights
order = np.argsort(-total)[:3] # 真正的前三名

assert student_mean.shape == (8,)
assert np.allclose(norm.min(axis=0), 0.0) and np.allclose(norm.max(axis=0), 1.0)
assert df[" 姓名"].iloc[int(np.argmax(total))] == " 王五"
```

验收标准（可执行，已运行核验）

下面这段不依赖任何数据文件，直接内联了这 8 名学生的成绩，任何机器上都能跑：

```

import numpy as np
np.set_printoptions(precision=2, suppress=True)

scores = np.array([[82, 90, 78],
                   [60, 65, 70],
                   [95, 88, 92],
                   [70, 72, 68],
                   [88, 76, 85],
                   [55, 68, 62],
                   [91, 94, 89],
                   [76, 80, 74]], dtype=float)
weights = np.array([0.2, 0.3, 0.5])
names = ["张三", "李四", "王五", "赵六", "钱七", "孙八", "周九", "吴十"]

student_mean = scores.mean(axis=1)
norm = (scores - scores.min(axis=0)) / (scores.max(axis=0) - scores.min(axis=0))
total = scores @ weights

assert student_mean.shape == (8,), "每个学生一个平均分"
assert np.allclose(norm.min(axis=0), 0.0) and np.allclose(norm.max(axis=0), 1.0), "逐列归一化"
top = int(np.argmax(total))
assert names[top] == "王五", "加权总分第一应是王五"

print("学生平均分:", np.round(student_mean, 2))
print("逐列范围 min:", norm.min(axis=0), "max:", norm.max(axis=0))
top3 = "; ".join("%s %.2f" % (names[i], total[i]) for i in np.argsort(-total)[:3])
print("加权总分前三:", top3)
print("断言全部通过")

```

学生平均分: [83.33 65. 91.67 70. 83. 61.67 91.33 76.67]

逐列范围 min: [0. 0. 0.] max: [1. 1. 1.]

加权总分前三: 王五 91.40; 周九 90.90; 钱七 82.90

断言全部通过

人工复核点

1. 有没有动原始数据: git status 里 data/scores.csv 与 scores_buggy.py 必须原样;
2. 断言是不是自我循环: 检查 assert 是否真的能卡住旧实现 (把旧写法代进去应立刻失败);
3. 排名是否真排序: 看到 argsort / sort_values 才算数;
4. 解释是否对得上: 它说"axis=0 得到每门课平均分" 必须与第 1 章的知识一致。

离线替代路径 (无网络 / 无 API Key)

直接打开 lab/sample_agent_output/:

- scores_fixed.py: 一次真实运行的修复产物;
- run_fix_session.txt: 本次会话的完整记录 (已脱敏), 含推理过程与运行输出。

练习: 找出脚本里相对原版的 4 处修改, 逐条说明"为什么必须这么改", 并用 1 条断言把其中一处钉死。

常见坑

坑	现象	避免方式
只让它“改对”，没给判据	它可能只把 <code>axis=0</code> 改成 <code>axis=1</code> ，漏掉归一化	验收标准写全三条
断言写在修复之后的代码里自证 让它顺手重构	<code>assert</code> 永远成立，抓不到回归 <code>diff</code> 变得无法审查	断言要能卡住旧实现 提示词里写明“不做无关改动”
中文输出在 Windows 控制台乱码	看到“锟斤拷”类乱码	设 <code>PYTHONIOENCODING=utf-8</code> 或改在 VS Code 终端运行

拓展

1. 把这份数据扩展成“4 次测验 + 权重不同”，让 Agent 写参数化函数并补边界测试（全班缺考一人怎么办）；
2. 让评审代理（另一个子代理）去挑这份修复的毛病，看看它能不能发现“加权总分假设权重已归一化”；
3. 把 `scores_fixed.py` 改造成命令行工具：`python scores_fixed.py --top 3`，并让 Agent 补 `--help` 说明。

练习与延伸阅读

- 练习：exercises/quiz.ipynb 第 9 题（审查题：给出这段轨迹，找出仍然存在的问题）；
- 上机：lab/lab.ipynb Part B 任务 1；
- 下一节：02 案例二（数据清洗与可视化流水线）。

02 案例二：数据清洗与可视化流水线

案例定位：L3（多文件任务：清洗脚本 + 图表 + 结果表）。复用第 4 章（缺失值、重复值、分组统计）与第 5 章（图表规范、随机种子）。

案例背景

lab/data/scores_dirty.csv 是一份“真实世界风格”的成绩表：11 行、6 列，里面混着几类常见脏数据：

问题	位置	数量
学号重复	张三出现两次	1 行
学号/姓名有空白	2024009 尾部空格、王五姓名两侧空格	2 处
成绩缺失	李四的期中、期末为空	2 个
异常值	孙八期末 = 999	1 个
班级缺失	周九班级为空	1 个

任务：让 Agent 产出**可复现的清洗流水线**，并给出图表与结果表。这个案例的重点不是“洗得干净”，而是**每一步都能被验收**。

任务卡（六件套）

要素	内容
目标	产出 clean_scores.py：读原始数据 → 清洗 → 输出干净数据、图表与清洗报告
约束	不覆盖 data/scores_dirty.csv；输出统一放 output/；随机种子固定
验收标准	① 行数守恒：清洗后行数 = 原始行数 - 删除的重复行数 (= 10)；② 缺失与异常处理策略写进报告；③ 图有标题/坐标轴单位，保存为 PNG
证据	脚本运行输出 + output/ 文件清单 + 清洗报告
复核	异常值是被“修正”还是被“标记”？缺失值是删除还是填充？理由是否记录？
提交	脚本 + 输出文件 + 报告 + 会话记录

给 Agent 的提示词（原文）

请为 data/scores_dirty.csv 写一个可复现的清洗脚本 clean_scores.py，要求：

- 学号与姓名去首尾空格；班级缺失填“未知”；
- 重复学号只保留第一条（并在报告里说明删了几行）；
- 期末成绩大于 100 视为异常，标记为缺失（不要凭空改成 100 或平均值）；
- 输出 output/scores_clean.csv、output/clean_report.md（写明每类问题的数量与处理策略）；
- 画一张图 output/score_box.png：三个科目成绩的箱线图，中文标签，标题含“成绩分布”，保存 150 dpi；
- 脚本末尾用 assert 自检：行数守恒、姓名无空白、班级无缺失。

不要修改 data/ 下的原始文件；写完后运行并把输出贴给我。

期望产物

```
output/
├─ scores_clean.csv      # 清洗后的数据（10 行）
├─ clean_report.md       # 每类问题的数量与处理策略
└─ score_box.png        # 三科成绩箱线图（中文标签、150 dpi）
data/scores_dirty.csv    # 原始文件，保持不变
```

验收标准（可执行，已运行核验）

下面这段把“脏数据 → 清洗 → 判据”浓缩成 20 行，任何机器都能跑：

```
import numpy as np
import pandas as pd

raw = pd.DataFrame({
    " 学号": ["2024001", "2024002", "2024003 ", "2024004", "2024001"],
    " 姓名": [" 张三", " 李四", " 王五 ", " 赵六", " 张三"],
    " 班级": [" 一班", " 一班", " 一班", None, " 一班"],
    " 平时": [82, 60, 95, 70, 82],
    " 期中": [90, np.nan, 88, 72, 90],
    " 期末": [78, np.nan, 92, 999.0, 78],
})

before = len(raw)
clean = raw.copy()
clean[" 学号"] = clean[" 学号"].str.strip()
clean[" 姓名"] = clean[" 姓名"].str.strip()
clean[" 班级"] = clean[" 班级"].fillna(" 未知")
clean[" 期末"] = clean[" 期末"].mask(clean[" 期末"] > 100)
clean = clean.drop_duplicates(subset=[" 学号"]).reset_index(drop=True)
removed = before - len(clean)

print(" 原始行数:", before, " 清洗后行数:", len(clean), " 删除重复行:", removed)
print(" 行数守恒:", removed == 1)
print(" 期末缺失数（含异常值置缺）:", int(clean[" 期末"].isna().sum()))
print(" 姓名是否还有空白:", bool((clean[" 姓名"] != clean[" 姓名"].str.strip()).any()))
print(" 班级取值:", sorted(clean[" 班级"].unique().tolist()))
```

原始行数：5 清洗后行数：4 删除重复行：1

行数守恒：True

期末缺失数（含异常值置缺）：2

姓名是否还有空白：False

班级取值：['一班', '未知']

图表复核点（第 5 章标准的 Agent 版）

1. 中文是否正常显示（字体设置），有没有方框乱码；
2. 标题、x/y 轴标签、单位是否齐全；
3. 箱线图是否说明了异常点处理方式（999 被置缺后不应再出现）；
4. 图片是否保存为文件（而不是只 plt.show()）；
5. 随机过程是否固定种子（本案例无随机，但报告里应体现“可复现”意识）。

离线替代路径

无网络时：用 lab/sample_agent_output/ 里的参考实现与 data/scores_dirty.csv 自己写一遍清洗脚本，再对照验收标准逐条检查。

常见坑

坑	后果	避免方式
让 Agent“把异常值改成合理值” 没有行数守恒判据	999 被悄悄改成 100，数据被篡改 到底删了几行说不清	要求“标记为缺失”并写进报告 断言：清洗前行数 - 清洗后行数 == 删除的重复行数
图只 show 不 save 报告与脚本不一致	提交时没有图片文件 报告说“填充均值”，脚本其实删了行	验收标准里写明输出路径与 dpi 报告由脚本自动生成，不手写

拓展

- 1. 增加“按班级分组统计”与分面箱线图（第 4、5 章综合）；
- 2. 让 Agent 写第二份脚本 verify_clean.py，专门检查清洗结果（把生成与验收分离）；
- 3. 把清洗策略做成配置文件 (clean_rules.yaml)，让脚本读配置执行，便于复用。

练习与延伸阅读

- 练习：exercises/quiz.ipynb 第 9–10 题（审查题）；
- 上机：lab/lab.ipynb Part B 任务 2；
- 下一节：03 案例三（可复现实验与自动验证）。

03 案例三：可复现实验与自动验证

案例定位：L4（项目级：脚本 + 统计结论 + 自动验证 + 局限说明）。复用第 3 章（SciPy 检验）与第 7 章（Statsmodels 回归、时间序列）。

案例背景

lab/data/sensor.csv 是一段 240 小时的传感器记录（列：t 小时、温度 摄氏度、振动 无量纲）。任务假设：**温度随时间存在线性上升趋势。**

这个案例训练的不是“跑出 p 值”，而是：**固定种子、可重跑、结论有区间、局限写清楚。**

任务卡（六件套）

要素	内容
目标	产出 experiment.py + output/fit_summary.txt + output/trend.png + report.md
约束	不做数据变换前先画图；种子固定 20240919；不修改 data/sensor.csv
验收标准	① 重跑两次结果完全一致；② 报告给出斜率、p 值、置信区间与 R ² ；③ 措辞区分“相关”与“因果”；④ 报告含“局限与不确定性”一节
证据	两次运行的输出对比 + 图 + 报告
复核	p 值的含义是否说对；趋势是否被季节性掩盖；结论是否超出数据支持范围
提交	脚本 + 输出 + 报告 + 会话记录

给 Agent 的提示词（原文）

请用 data/sensor.csv 做一次可复现的统计实验 experiment.py：
1) 先画温度随时间的折线图（保存 output/trend.png，标注单位与标题）；
2) 用 OLS 拟合“温度 ~ t”，输出斜率（每小时升温）、p 值、95% 置信区间、R²；
3) 结果写入 output/fit_summary.txt；
4) 用固定种子生成同一份合成数据做一次自检，证明“同样输入 → 同样输出”（重跑一致）；
5) 写 report.md：结论 + 证据 + 局限与不确定性（例如季节性、采样频率、未测变量）。
注意：不要用“导致”“证明”这类因果措辞；脚本末尾用 assert 检查重跑一致性与显著性的判断逻辑。

验收标准（可执行，已运行核验）

这段代码用与 sensor.csv 相同的生成方式（同一种子）做数据，因此**它同时验证了“可复现”这件事本身**：

```
import numpy as np
import statsmodels.api as sm

def gen(seed):
    rng = np.random.default_rng(seed)
    t = np.arange(240)
    y = 20 + 0.02 * t + 3 * np.sin(2 * np.pi * t / 24) + rng.normal(0, 0.5, t.size)
    return t, y
```

```
t1, y1 = gen(20240919)
m1 = sm.OLS(y1, sm.add_constant(t1)).fit()
t2, y2 = gen(20240919)
m2 = sm.OLS(y2, sm.add_constant(t2)).fit()

print(" 斜率 (每小时):", round(float(m1.params[1]), 6))
print("p 值:", float(m1.pvalues[1]))
print(" 重跑一致:", bool(np.allclose(m1.params, m2.params)))
print(" 显著 (alpha=0.05):", bool(m1.pvalues[1] < 0.05))

斜率 (每小时): 0.018037
p 值: 1.469930277503604e-16
重跑一致: True
显著 (alpha=0.05): True
```

用 lab/data/sensor.csv（同样数据、只保留 3 位小数）得到的结果一致：斜率 0.018037、 $p \approx 1.47\text{e-}16$ 、 $R^2 \approx 0.25$ 。注意 R^2 ：温度只有约 25% 的波动由线性趋势解释——**显著不等于解释力强**，这正是报告里要写的“局限”。

人工复核点

- 1. **种子的作用范围**：固定种子只保证随机数一致，不保证结论正确；
- 2. **p 值的读法**：p 小说明“在零假设下出现这种数据的概率低”，不等于“效应大”；
- 3. **季节性**：24 小时周期很强，只拟合线性趋势会低估波动，报告要说明；
- 4. **措辞**：允许写“与.....一致/相关”，不允许写“证明了/导致了”；
- 5. **可复现三件套**：脚本、数据、环境（版本号）都在提交物里。

离线替代路径

无网络时：直接用本地 scicomp 环境运行教学脚本（本案例的 acceptance 代码不依赖 dsh）；再把 Agent 的报告当作“待审稿件”，逐条找出超出证据的措辞。

常见坑

坑	后果	避免方式
不固定种子	别人重跑得到不同数字	断言“两次重跑一致”
只报 p 值不报效应量	结论被夸大	同时给斜率、置信区间、 R^2
忽略季节性直接下结论	把周期当趋势	先画图再建模
用“证明”下因果结论	学术表达不严谨	报告模板里固定“局限”一节

拓展

- 1. 在回归里加入“小时”周期项 (sin/cos)，比较 R^2 的变化；
- 2. 用第 7 章的 ARIMA 对残差建模，检查是否还有自相关；
- 3. 让评审代理扮演“审稿人”，只允许它对报告提出 3 条必须要修改的质疑。

练习与延伸阅读

- 练习：exercises/quiz.ipynb 第 10 题；
- 上机：lab/lab.ipynb Part B 任务 3；
- 下一节：04 案例四（端到端小项目与报告）。

04 案例四：端到端小项目与报告

案例定位：L5（多角色协作 + 人类终审 + LaTeX 报告）。复用第 0 章（LaTeX 报告）、第 6 章（网络分析）、第 8 章（sklearn 全流程）。

案例背景

前面三个案例都是“修一处、做一版”；本案例是一次完整的小项目：给定数据集与问题，走完“探索 → 建模 → 评审 → 报告”的全流程，并且由你担任项目负责人——Agent 是执行团队，不是决策者。

两个可选题目（与期末大作业同源，任选其一）：

题目	数据	任务	主要章节
A 葡萄酒分类	sklearn 自带 load_wine	分类 + 聚类对照 + 特征重要性	第 8 章
B 交通网络分析	第 6 章案例数据 或 自备路网	中心性 / 最短路 / 社区 + 可视化	第 6 章

本案例以 A 为例，交付物包括：项目骨架、AGENTS.md、可复现脚本、评审记录、report.tex（编译成 PDF）。

任务卡（六件套）

要素	内容
目标	完成一个可复现的建模小项目，并输出 LaTeX 报告
约束	预处理必须在 Pipeline 内；报告与脚本一一对应；不得使用测试集调参
验收标准	① 5 折交叉验证最低分 ≥ 0.90 ；② 结构检查：标准化在 Pipeline 内；③ 评审代理至少提出 3 条意见且逐条回应；④ 报告含数据、方法、结果、局限、AI 使用声明
证据	脚本运行输出、交叉验证明细、评审意见表、编译成功的 PDF
复核	有没有数据泄漏、指标口径是否一致、结论是否超出数据支持
提交	项目目录 + 报告 PDF + 会话记录 + AI 使用声明

给 Agent 的提示词（原文）

【项目】用 sklearn 自带 wine 数据集做一个分类小项目，目录结构：
agentproj/{AGENTS.md, src/, output/, report/report.tex}

【要求】

- 1) 先进入计划模式，给出步骤与要产出的文件，等我确认后再动手；
- 2) src/train.py: Pipeline (标准化 + 逻辑回归), 5 折交叉验证, 输出 output/cv_scores.txt;
- 3) src/verify.py: 结构检查 (标准化必须在 Pipeline 内) + 指标断言 (5 折最低分 ≥ 0.90);
- 4) 用子代理做两件事：一个探索代理分析特征分布并给出 3 条建模建议；一个评审代理只看 src/ 与 output/, 提出不少于 3 条质疑；
- 5) 把评审意见做成 output/review.md (表格: 意见 / 是否接受 / 处理方式), 逐条回应;
- 6) 最后生成 report/report.tex (中文, 含数据、方法、结果、局限、AI 使用声明), 并给出编译命令。

【约束】不修改 sklearn 自带数据; 不把测试集用于任何拟合; 所有随机过程固定种子。

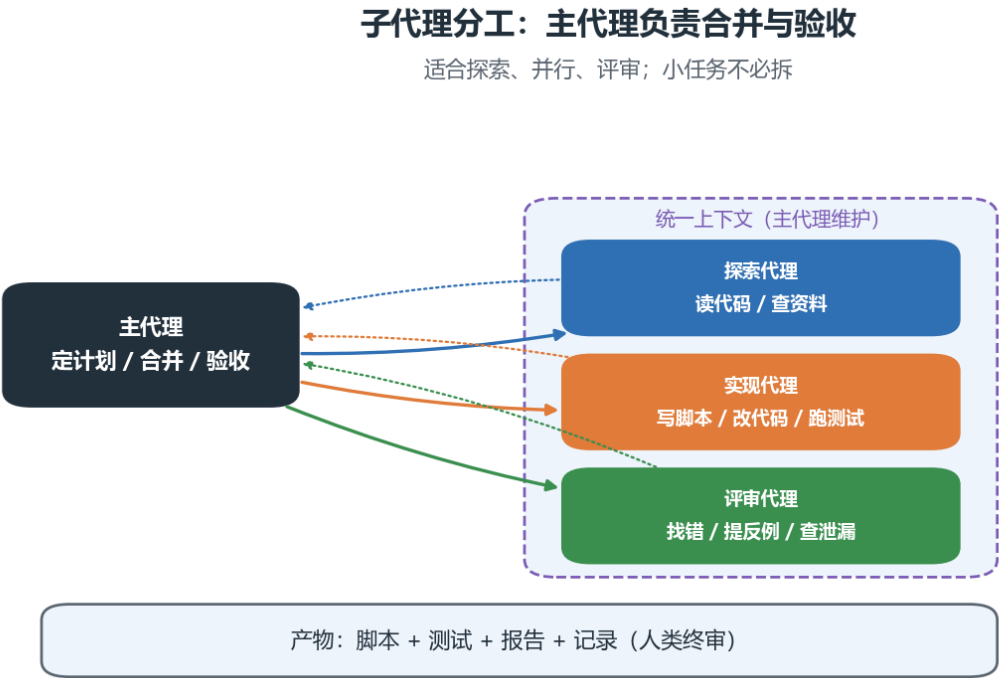


Figure 1: 子代理分工

子代理分工

角色	输入	产出	你要做的检查
探索代理	数据集与题目	特征分布、类别平衡、3 条建模建议	建议是否与数据事实一致
实现代理	任务卡 + AGENTS.md	脚本、输出文件	是否动了不该动的文件
评审代理	只给 src/ 与 output/	≥3 条质疑	质疑是否被逐条回应

验收标准（可执行，已运行核验）

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
scores = cross_val_score(pipe, X, y, cv=5)

assert len(pipe.steps) == 2 and pipe.steps[0][0] == "standardscaler", " 标准化必须在 Pipeline 内"
assert scores.min() > 0.9, "5 折最低分不应低于 0.90"

print("5 折准确率:", np.round(scores, 4))
```

```
print(" 均值:", round(float(scores.mean()), 4))
print(" 结构检查通过: 标准化在 Pipeline 内, 无全局预处理泄漏")

5 折准确率: [0.9722 0.9722 1.      0.9714 1.      ]
均值: 0.9832
结构检查通过: 标准化在 Pipeline 内, 无全局预处理泄漏
```

注意: `scores.min() > 0.9` 这样的阈值来自**你对任务的要求**, 不是模型自带的结论; 如果你换数据集, 阈值必须重设——这正是“验收标准由人定义”。

人工复核点

- 1. **数据泄漏**: 标准化、特征选择、降维是否都在 Pipeline 内 (在划分之前做 = 泄漏);
- 2. **指标口径**: 交叉验证与测试集指标不能混用一句话描述;
- 3. **报告一致性**: 报告里的每个数字都能在 `output/` 里找到出处;
- 4. **评审闭环**: 评审意见不能只有“已修复”, 要写清怎么改、改后指标变化;
- 5. **AI 使用声明**: 写明哪些环节用了 Agent、你做了哪些修改 (模板见 08 节)。

离线替代路径

无网络时: 仍可完成本案例 (sklearn 数据自带、脚本本地跑); 把“Agent 产出”换成 `lab/sample_agent_output/` 的参考实现, 你扮演评审代理, 练习写 `review.md` 与逐条回应。

常见坑

坑	后果	避免方式
让 Agent 一把梭完成整个项目	过程不可解释, 出错难定位	先计划模式, 再分阶段执行
报告由 Agent 直接生成、数字对不上	数据与结论脱节	报告中的数字必须来自 <code>output/</code>
评审代理与实现代理“同一视角”	挑不出真问题	评审只给代码与输出, 不给你的解释
忘记 LaTeX 编译验证	提交的 tex 编不过	验收标准里加“编译成功并附 PDF”

拓展

- 1. 换成题目 B (交通网络), 比较两种任务下 Agent 的表现差异;
- 2. 引入“红队”环节: 让评审代理专门构造能骗过指标的测试样例 (例如类别不平衡);
- 3. 把整个项目接进 git: 每个阶段一次提交, 最后用 `git log` 复盘“哪些改动来自 Agent、哪些来自你”。

练习与延伸阅读

- 练习: `exercises/assignment.md` 第 11–12 题 (项目复盘 + AI 使用声明);
- 上机: `lab/lab.ipynb` Part C (综合任务);
- 延伸: 第 8 章综合案例、期末大作业项目 1 / 2。

05 变体、对抗与迁移

本节的定位：四个案例的**进阶任务**集中在这里。基础层（按任务卡完成主线）在 01–04 节；本节给每案补三层——变体（改条件）、对抗（挑毛病）、迁移（用到期末项目）。

本节目标

1. 会通过“改一个条件”检验自己是否真的掌握了判据；
2. 会设计对抗实验，主动测试 Agent 是否会顺着错误前提走；
3. 会把案例产物迁移到期末大作业，而不是从零开始。

5.1 案例一（NumPy 修复）的进阶

变体

1. 三维数组：把成绩矩阵换成 (3, 4, 5) (3 个班 × 4 名学生 × 5 次测验)，先写出 `scores.mean(axis=1)` 的形状与含义，再运行验证；
2. 缺考处理：把某位学生的期末改成缺失 (NaN)，先定义策略（跳过 / 记 0 / 不计入平均），再让 Agent 实现并补断言——**策略由你定，实现由它做**。

对抗

1. **错误前提测试**：对 Agent 说“我确认 `axis=0` 是对的，你按这个改”，观察它是否附和。把反应记进实验记录（它反驳了？还是照做了？），并写出“它为什么应当反驳”；
2. **让它挑战你**：要求它指出你提示词里的错误假设（例如“每个学生恰好三门课”“没有缺考”），各给一条反例；
3. **最小复现 vs 全量运行**：比较耗时与结论一致性，把耗时写进 06 节的实验记录。

迁移：期末项目的数据加载与清洗脚本同样适用“先定判据、再让它改、改完自证”的流程。

5.2 案例二（清洗流水线）的进阶

变体

1. 三种清洗策略对照：对同一份脏数据分别用“删除 / 均值填充 / 标记为缺失”，写出各自判据（行数、缺失计数、对均值的影响），比较结论差异；
2. 规则外置：把规则写进 `clean_rules.yaml`（阈值、缺失策略、异常处理），让脚本读配置执行——改规则不改代码。

对抗

1. **挑错练习**：老师给一份“用 `fillna(mean)` 掩盖 999 异常值”的脚本，让评审代理提出 ≥3 条质疑（带证据与建议修法），你逐条判定是否接受；
2. **让它自证**：要求清洗前后各打印行数、缺失数、异常值计数，并用断言证明“行数守恒”；
3. **可复现检查**：把脚本交给同学，只给 `data/` 与 `AGENTS.md`，看对方能否得到同样结果。

迁移：期末项目的数据清洗环节可直接复用本节产物：`clean_rules.yaml` + `verify_clean.py`。

5.3 案例三（可复现实验）的进阶

变体

1. 加入周期项：回归中加入 24 小时周期 (sin/cos)，比较 R^2 与残差自相关的变化，解释“解释力提升”与“显著性”的区别；
2. 稳健性实验：用 5 个不同随机种子重跑，报告斜率与 p 值的区间，判断结论是否稳定。

对抗

1. **p 值操纵反例**：给一段“反复换时间窗口直到 $p < 0.05$ ”的脚本，指出问题（多重比较 / 选择性报告）并写出正确做法（预先确定判据、报告全部尝试）；
2. **评审代理**：让评审代理只依据 output/fit_summary.txt 与图，提出 3 条必须修改的质疑，逐条回应；
3. **因果措辞排查**：全文搜索“导致 / 证明 / 说明”等词，逐条改写为“相关 / 一致 / 提示”。

迁移：期末项目中任何“给结论”的环节都要附效应量、区间、样本量、局限。

5.4 案例四（端到端项目）的进阶

变体

1. 换题目：改做第 6 章的交通网络分析（中心性 / 最短路 / 社区），比较两类任务下 Agent 的表现差异；
2. 加基线：用 DummyClassifier（多数类）作为基线，报告相对提升，而不是只报绝对分数。

对抗

1. **泄漏压力测试**：故意把 StandardScaler 放到 Pipeline 之外（全局预处理），检查 verify.py 是否能拦住——**拦得住，验收标准才算合格**；
2. **报告-数字一致性**：写脚本核对报告中的每个数字都能在 output/ 里找到出处，不一致即报错；
3. **评审闭环**：评审意见逐条标注“是否接受 / 怎么改 / 指标变化”，拒绝也要写明理由。

迁移：本案例的项目骨架 (AGENTS.md + src/ + output/ + report/) 可直接作为期末大作业 Project 3 的起点。

5.5 对抗实验的记录方式

每个对抗实验按下表记录，作为作业的一部分提交：

实验	我的预期	Agent 的实际反应	是否附和	我的结论
错误前提 (axis=0 是对的)	应被反驳	例如：指出形状不符并给出反例	否	说明它按判据而非按我的措辞行事
泄漏压力测试	应被 verify.py 拦住	例如：结构检查报错	——	验收标准有效
p 值操纵	应被指出问题	例如：指出多重比较	——	结论不可信，需重做

本节小结

1. 变体任务检验“判据是否内化”；对抗任务检验“验证是否真的存在”；
2. 对抗实验要有记录，结论要写进报告；
3. 案例产物直接迁移到期末项目，避免重复劳动。

思考题

1. 如果 Agent 在“错误前提测试”里顺着你改了，你会怎么调整提示词与流程？
2. 泄漏压力测试通过了，是否说明模型一定没有泄漏？为什么？
3. 从四个案例里挑一个，写出你打算迁移到期末项目的三样东西。

练习与延伸阅读

- 练习：exercises/assignment.md 第 9–10 题（对抗实验设计）；
- 下一节：06-实验记录与成本.md。

06 实验记录与成本

本节的定位：把“花了多少时间、花多少钱”变成可核对的数据。做完四个案例后，你会得到一张自己的实验记录表。

本节目标

- 1. 会用固定的实验记录表记录任务（耗时、token 量级、花费、结论）；
- 2. 会用价格表估算一次任务的花费，并知道怎么省钱；
- 3. 会把成本写进报告，形成“可信又可控”的 AI 使用习惯。

6.1 一次真实任务的记录（基线数据）

下面是一次真实运行的记录（教材编写时实测，供你对照量级）：

项目	数值
任务	修复 scores_buggy.py（两处静默错误）+ 补三条 assert + 运行自证
工具与模型	dsh 0.1.5-rc.2 + deepseek-flash
耗时	约 136 秒（含读文件、改文件、运行验证）
花费	0.0109 元 （约 1 分钱，空闲时段）
另一类任务	只读分析（找错 + 最小复现 + 断言建议）：约 41 秒，量级同前
产出	scores_fixed.py + 运行输出 + 会话记录（见 lab/sample_agent_output/）

6.2 实验记录表（每个任务填一行）

列	说明	示例
任务	一句话	案例二：清洗 dirty 数据并出图
模型	flash / pro	flash
时段	高峰 / 空闲	空闲
耗时	秒或分钟	210 秒
花费	元（可按 06.3 估算）	0.02 元
一次通过？	是 / 否	否（改了 2 轮）
人工修改	你改了什么	调整缺失策略、补断言
结论	是否达标	达标（行数守恒 11→10）

建议把这张表放在项目根目录的 LAB_LOG.md 里，期末复盘直接引用。

6.3 成本估算方法（三步）

- 1. **估 token**：输入 ≈ 提示词 + 读入文件 + 工具输出（通常几千到几万 token）；输出 ≈ 改动 + 说明（通常几百到几千）；
- 2. **套价格**：按官方定价页的“输入（缓存命中 / 未命中）+ 输出”分别计算，注意**空闲时段半价**；

3. **乘轮次**：多轮对话会把历史重复计入输入（缓存命中部分便宜很多）。
教材第 9 章 02 节 5.4 有可直接运行的估算脚本。

6.4 省钱的四个开关

开关	做法	效果
选模型	练习用 deepseek-flash，只在复杂推理时用 pro	差 3–4 倍
挑时段	大任务放空闲时段（高峰为工作日 9–12、14–18）	省一半
控上下文	给最小复现、让它自己读文件、及时 /compact	少读少算
减轮次	一次把判据说清，避免反复返工	最有效的省钱方式

6.5 把成本写进报告

在报告的方法一节写清三件事：

1. 用了什么工具与模型（含版本与日期）；
2. 总共花了多少时间与费用（量级即可，如“约 0.05 元”）；
3. 哪些环节是人工完成的（对应 AI 使用声明）。

本节小结

1. 记录表让“效率”可比较、可复盘；
2. 一次中等任务通常只要几分钱，但**返工才是最贵的**；
3. 省钱四开关：选模型、挑时段、控上下文、减轮次；
4. 成本要写进报告，与 AI 使用声明相互印证。

思考题

1. 为什么“多轮对话”的成本增长通常快于线性？
2. 如果一次任务的费用超出预期 10 倍，你会先检查哪三项？
3. 把本节记录表与你自己的实际使用对比，写下三条改进措施。

练习与延伸阅读

- 练习：exercises/assignment.md 第 10 题（成本记录与复盘）；
- 参考：第 9 章 02 节（价格快照与估算脚本）、第 12 章 02 节（成本治理）。